

SIMATS ENGINEERING
ASSESSMENT TOOL 1
COURSE CODE: CSA1406
COURSE NAME: COMPILER DESIGN

Student Name: v a i s h n a v . m

Register Number: 192524251

CO5: Develop code optimization and Code Generation using DAG representation with data flow analysis to produce optimized target code.

Q1. Intermediate Code Generation and Control Flow Analysis

Given code:

```
int sum = 0;
for (i = 0; i <= 10; i++)
    sum = sum + a[i];
```

(a) Three Address Code (TAC)

1. sum = 0
2. i = 0
3. L1: if i > 10 goto L2
4. t1 = a[i]
5. t2 = sum + t1
6. sum = t2
7. t3 = i + 1
8. i = t3
9. goto L1
10. L2: end

Initialization: sum and i are initialized to zero. The loop condition is checked before every iteration. The array element a[i] is loaded into a temporary variable, added to sum, and then i is incremented.

(b) Basic Blocks and CFG

Leader statements: 1, 2, 3, 4, and 10.

Basic Block B1: $\text{sum} = 0$

Basic Block B2: $i = 0$

Basic Block B3: L1: if $i > 10$ goto L2

Basic Block B4: $t1 = a[i]$; $t2 = \text{sum} + t1$; $\text{sum} = t2$; $t3 = i + 1$; $i = t3$; goto L1

Basic Block B5: L2: end

CFG edges: $B1 \rightarrow B2 \rightarrow B3$. From B3, the true branch goes to B5 and the false branch goes to B4. B4 loops back to B3.

Q2. Intermediate and Target Code Generation

Expression: $x := (a + b)(c - d) + ((e / f) * (a + b))$

Assuming the juxtaposition $(a + b)(c - d)$ denotes multiplication.

(a) Three Address Code

$t1 = a + b$

$t2 = c - d$

$t3 = t1 * t2$

$t4 = e / f$

$t5 = t4 * t1$

$t6 = t3 + t5$

$x = t6$

The common sub-expression $(a + b)$ is evaluated once as $t1$ and reused in the second part.

(b) Target Code

MOV R1, a

ADD R1, b ; $R1 = a + b$

MOV R2, c

SUB R2, d ; $R2 = c - d$

MUL R2, R1 ; $R2 = (a+b)(c-d)$

MOV R3, e

DIV R3, f ; $R3 = e/f$

MUL R3, R1 ; $R3 = (e/f)(a+b)$

ADD R2, R3 ; $R2 = \text{final result}$

MOV x, R2

Q3. DAG Construction and Optimized Code

Basic block:

$d := b * c$

$e := a + b$

$b := b * c$

$a := e - d$

(a) DAG Analysis

The multiplication $b * c$ occurs twice. Therefore, one multiplication node can represent the common sub-expression $b * c$. The first result is assigned to d and the same computed value can be reused for the assignment to b .

DAG nodes: b, c, a are leaf nodes. Node $N1$ represents $b * c$ and has labels d and b . Node $N2$ represents $a + b$. Node $N3$ represents $e - d$, with e and d referring to their corresponding values.

(b) Optimized Code Using One Register

LOAD R, b

MUL R, c

STORE d, R

STORE b, R

LOAD R, a

ADD R, b

STORE e, R

LOAD R, e

SUB R, d

STORE a, R

The common multiplication $b * c$ is performed only once and its value is reused, reducing redundant computation.

Q4. Optimal Code Generation Under Register Constraints

Expression: $(a + b) - (c - (d + e))$

(a) Using Two Registers

MOV R1, d

ADD R1, e ; $R1 = d + e$

MOV R2, c

SUB R2, R1 ; $R2 = c - (d+e)$

MOV R1, a

ADD R1, b ; $R1 = a + b$

SUB R1, R2 ; $R1 = (a+b) - (c-(d+e))$

(b) Using One Register

MOV R, d

ADD R, e

STORE T1, R

MOV R, c

SUB R, T1

STORE T2, R

MOV R, a

ADD R, b

SUB R, T2

STORE result, R

With one register, intermediate results must be stored in memory temporaries so that the register can be reused.

Q5. Code Optimization

Given C++ code:

```
void optimizeCode(int n, int x, int y, int z, int w, int* a) {  
    for (int i = 0; i < n; i++) {  
        a[i] = x * y + z;  
        a[i + n] = x * y + w;  
    }  
}
```

(a) Common Subexpression Elimination and Code Motion

The expression $x * y$ is loop-invariant because x and y do not change inside the loop. It is also a common sub-expression in both assignments. Therefore, it can be computed once before the loop.

Optimized version:

```
void optimizeCode(int n, int x, int y, int z, int w, int* a) {  
    int xy = x * y;  
    for (int i = 0; i < n; i++) {  
        a[i] = xy + z;  
        a[i + n] = xy + w;  
    }  
}
```

(b) Analysis and Benefits

1. Reduction in computations: $x * y$ is computed once instead of twice per loop iteration.
2. Runtime efficiency: Removing repeated multiplication reduces the work performed during every iteration.
3. Effect on loop performance: The loop contains fewer arithmetic operations, improving execution efficiency, especially for large values of n .

Overall, common subexpression elimination and loop-invariant code motion produce a more efficient version without changing the program's result.

Conclusion

The simulation demonstrates TAC generation, basic block formation, CFG construction, DAG-based optimization, register-aware target code generation, common subexpression elimination, and loop-invariant code motion. These techniques reduce redundant computations and improve target-code efficiency.